Classification 5G base stations

2025-04-28

Emma MENON

Contents

1. Introduction . . . 1
2. Problem description . . . 1
3. Data treatment . . . 2
   3.1. Data examination . . . 2
   3.2. Data preprocessing . . . 2
4. First model . . . 2
5. Data augmentation and model improvements . . . 4
   5.1. Data augmentation . . . 4
   5.2. Additional convolutional layer . . . 4
   5.3. L2 regularization and dropout . . . 4
6. Impact of modifications on model performance . . . 5
7. Hyperparameter tunning . . . 5
8. Result on Kaggle . . . 7
9. Conclusion . . . 7

1. Introduction

We are all using our cell phones on a daily basis. These are our connections with family and friends, help us to search for information, etc. The cell phone needs always be connected to the network, through a so-called base station (as telecommunication engineers, we call them eNodeB's or gNodeB's). But, there can be attackers taking advantage of security vulnerabilities in mobile networks. With the use of specialized hardware and software, they can steal some informations, send malicious messages, or track users. One of the known tools is a so-called False Base Station (FBS), sometimes called Rogue Base Station (RBS), or International Mobile Subscriber Identifier (IMSI) Catcher. This is a device pretending as the legitimate base station and trying the mobile terminals to connect. The false base station thus needs to behave as the legitimate one, i.e., it needs to broadcast the same information to the radio channel.

2. Problem description

The frame structure of 4G/LTE signal sent from the base station is shown on left part of Figure 1. A crucial part of transmitted information are so-called synchronization sequences. There are two of them - a primary synchronization sequence (PSS) and a secondary synchronization sequence (SSS). During our measurement campaign, we captured huge number of such sequences and used them to calculate the channel frequency responses. From these channel frequency responses, we created samples that you will process. An example of one sample visualized in 2D is shown in the right part of Figure 1. Note it consists of 72 rows (corresponding to 72 subcarriers allocated to PSS/SSS) and 48 columns (corresponding to 48 repetitions of channel frequency responses).

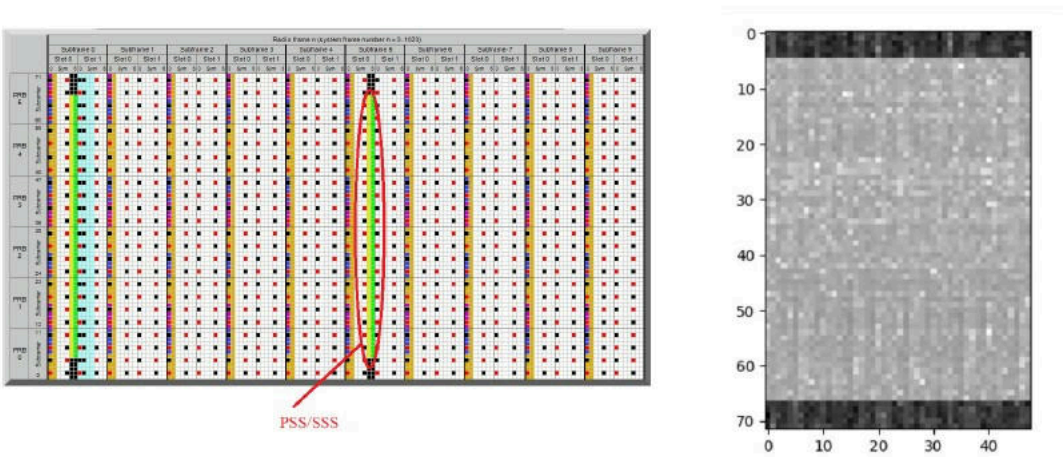

Figure 1: LTE/4G frame structure (left), example of training sample in 2D (right)

The goal of the project is to classify whether there is only a legitimate base station (gNodeB) of T-mobile operator (class 0) transmitting from a neighboring building or whether an attacker brought his transmitter into the building and is trying to steal the information from users (class 1 and class 2). The false/attacker's base station could be located at one of two positions - class 1 corresponds to the first one, and class 2 corresponds to the second one.
You can access the notebook with the code here.

3. Data treatment

3.1. Data examination

The data I worked with consists of 2D images, each represented as a matrix of 72 rows and 48 columns. The target variable is a categorical label with possible values 0, 1, or 2. In total, I have 1491 labeled samples (image categories) to train our model. However, the dataset is strongly imbalanced: there are 1209 samples labeled as 0, and 141 samples each labeled as 1 and 2, which correspond to approximately 81% and 9.5% of the data, respectively. Due to the imbalance in the dataset, technique such data augmentation could later be considered to improve model performance.
To ensure that the model I implemented learns effectively and generalizes well to new, unseen data, I split the dataset into training and testing sets.

3.2. Data preprocessing

Before implementing the model, I need to prepare the data properly. Machine learning algorithms do not perform well if numerical attributes have very different scales so I normalize the input values to fall between 0 and 1 by dividing each pixel value by 255, as pixel intensities in gray scale images range from 0 to 255.
Since the target variable is categorical, I apply one-hot encoding. This process creates a new column for each class, where a 1 indicates the presence of that class and a 0 indicates its absence. In the project case, the one-hot encoding can be schematized as shown in Figure 2

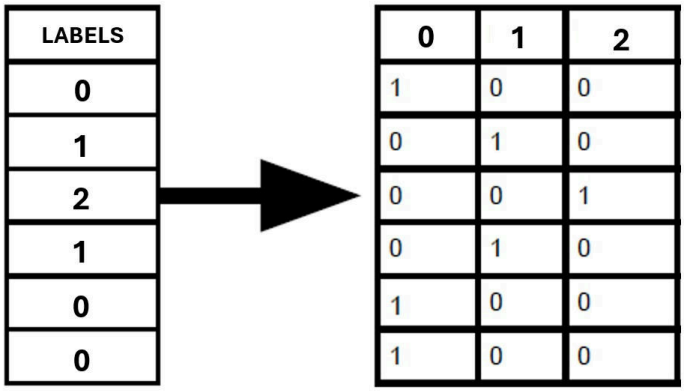

Figure 2: one hot encoding schematized

4. First model

To address this problem, I chose to use a Convolutional Neural Network (CNN) because of its strong performance in image classification tasks. Initially, I implemented a simple model to test the dataset, based on the architecture I finalized during the CNN lab. The model structure was as follows:

- One convolutional layer with 32 filters, a 3x3 kernel, and ReLU activation
- One max pooling layer to reduce the spatial dimensions
- One flattening layer to prepare the data for the fully connected layers
- One fully connected (dense) layer with 128 neurons and ReLU activation
- One output layer with softmax activation for multi-class classification

After defining the model, I compiled and trained it using the training and testing split, with the following parameters:

Parameter	**Value**
Loss function	categorical_crossentropy
Optimizer	Adam
Metrics	accuracy
Epochs	20
Batch size	100

At the end of the training, the model achieved an accuracy of 98,21%, which is a very strong result for an initial, "basic" model. However, it is important to monitor for potential overfitting, as very high accuracy on the training set does not always guarantee good generalization to new data. To better visualize the model's performance, the confusion matrix is shown in Figure 3

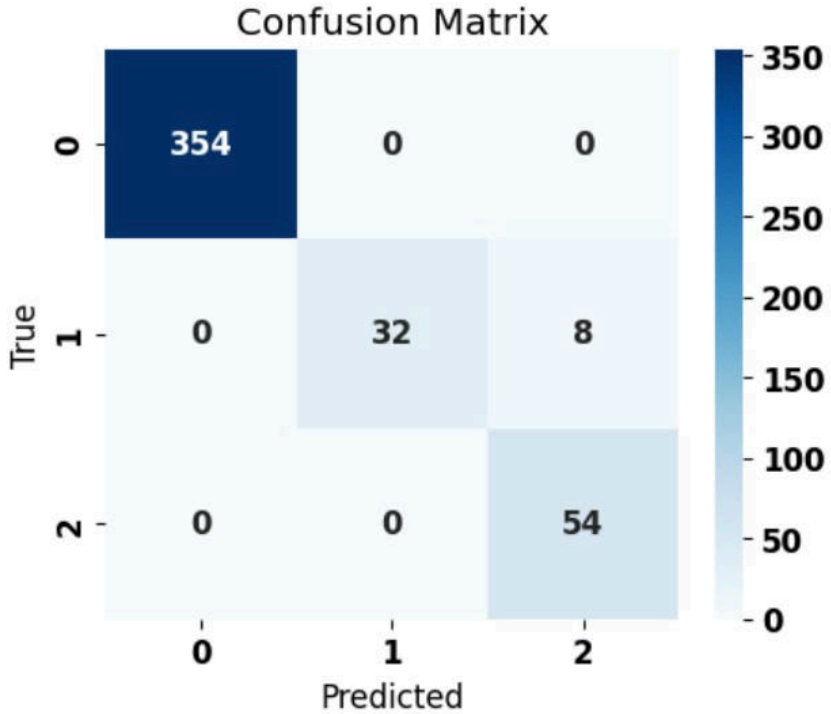

Figure 3: Confusion matrix for the first CNN model

From this confusion matrix, we can observe that the misclassification occurs primarily within the same class—8 instances of class 1 have been incorrectly classified as class 2. This could be attributed to the significant class imbalance in the dataset.
The next step is to further improve the model's performance using various techniques, even though it is already achieving satisfactory results.

5. Data augmentation and model improvements

5.1. Data augmentation

Given the limited size of the original dataset, data augmentation techniques were applied to artificially increase the number of training samples. Data augmentation helps to prevent overfitting and improve the generalization ability of the model by introducing variations in the training data. Specifically, each image in the training set was augmented through horizontal flipping, vertical flipping, and a combination of both. These transformations create new samples while preserving the original class labels, allowing the model to learn more robust and invariant features.
The choice of horizontal and vertical flipping was motivated by the nature of the images, where such transformations do not alter the semantic meaning of the classes. This selection ensures that the augmented data remains representative of the original distribution while improving the diversity of the training set.

5.2. Additional convolutional layer

To improve both the model's performance and efficiency, an additional convolutional layer was introduced. This layer, followed by a max-pooling operation, helps reduce the spatial dimensions of the feature maps earlier in the network. As a result, the model processes smaller feature maps in the later layers, which not only speeds up training and inference but may also improve the ability to extract meaningful features from the images. This modification strikes a balance between improving classification accuracy and maintaining computational efficiency, making the model faster and potentially more effective in generalizing to unseen data..

5.3. L2 regularization and dropout

To further improve the model's performance and prevent overfitting, I applied two regularization techniques: L2 regularization and dropout.
L2 regularization penalizes large weights by adding a term to the loss function that is proportional to the sum of the squared values of the weights. This encourages the network to keep the weights small, which helps in reducing the model's sensitivity to small variations in the input data. As a result, the model becomes smoother and more generalizable, improving its ability to perform well on unseen data.
Dropout is another regularization method where, during each training iteration, a random subset of neurons in a layer is temporarily ignored. This prevents the model from becoming overly reliant on specific neurons and encourages the network to develop redundant representations. By promoting more robust feature learning, dropout contributes to better generalization and helps mitigate the risk of overfitting.
For the initial experiments, a dropout rate of 0.5 and an L2 regularization factor of 0.001 were applied.

6. Impact of modifications on model performance

Step	Modification Applied	Accuracy
1	Basic model	98,21%
2	Data augmentation (flips)	97,99%
3	Additional convolutional layer	97,99%
4	L2 regularization and dropout	98,21%

First, data augmentation did not lead to an improvement in accuracy, but I have chosen to continue using it, as I strongly believe it will contribute to better generalization and help prevent overfitting. Adding a convolutional layer did not improve accuracy either, though this may be due to suboptimal parameter combinations. I plan to revisit this and test with more suitable parameters to see if it enhances performance. Then, the regularization techniques provided a slight improvement in accuracy, indicating that they are beneficial and should remain part of the model.

7. Hyperparameter tunning

Choosing the right hyperparameters is crucial for optimizing machine learning model performance. By finding the best values for the hyperparameters, the model can be trained more effectively and achieve better accuracy on unseen data. Hyperparameter tuning involves trying out different combinations of hyperparameters and evaluating the model's performance on a validation set. To find the best combination, I implemented the grid search algorithm. Grid search is a brute-force approach that systematically explores all possible combinations of hyperparameters within a predefined range. I chose this method because it is straightforward, systematic, and relatively easy to implement. The range of parameters I explored was :

Parameters	Range
number of neurons	$[150, 175, 200, 225]$
dropout rates	$[0.2, 0.4, 0.6]$
L2 regularization rates	$[0.001, 0.01, 0.1]$
number of layers	$[1, 2]$

It is worth noting that the number of epochs was not included as a hyperparameter in the grid search. This decision was made because the grid search already took a considerable amount of time to run with the selected parameter ranges. Additionally, based on preliminary experiments, I observed that the best model accuracy was typically achieved after around 10 epochs when using data augmentation. During the grid search, however, the number of epochs was intentionally set slightly higher (15) to ensure that models had sufficient time to converge. Final epoch selection was refined afterward by analyzing the validation performance curves.

Moreover, throughout the grid search, I plotted the training and validation curves and saved the training history for each run. This allows me to verify whether the best hyperparameter combination identified by the grid search is indeed the most optimal. Specifically, by inspecting the curves, I can quickly check for signs of overfitting and determine whether a better validation accuracy was achieved at an earlier epoch. In such cases, selecting an earlier stopping point could lead to better generalization.

In the end, the best set of hyperparameters found was :

Parameters	**Value**
number of neurons	175
dropout rate	0.2
L2 regularization rate	0.001
number of layers	2

Next, I need to fine-tune the model by selecting the optimal number of epochs to prevent overfitting. To do so, I have plotted the accuracy and loss against the number of epochs, as shown in Figure 4.

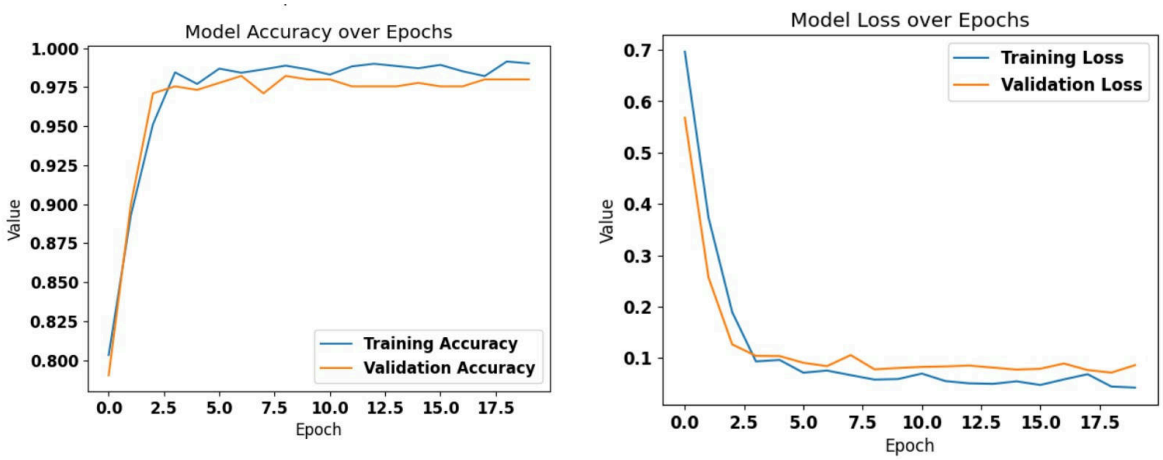

Figure 4: Model accuracy and loss over epochs

Based on these two graphs and the history provided in the notebook, it is clear that the optimal number of epochs for achieving the minimum loss and best accuracy is 9.

8. Result on Kaggle

To summarize, the model I used to classify the test data and make my submission on Kaggle is as follows:

Parameter	**Value**
Loss function	categorical_crossentropy
Optimizer	Adam
Metrics	accuracy
Epochs	9
Batch size	100
number of neurons	175
dropout rate	0.2
L2 regularization rate	0.001
number of layers	2

With this configuration, I achieved an accuracy of 96.66% on Kaggle, which I believe is a strong result for unseen data, demonstrating the model's generalization capabilities.
When comparing this with the training and validation accuracy, I obtained 98.78% for training and 97.99% for validation. The relatively small difference between these values suggests that the model is not overfitting, as it is performing well both on the training data and on new, unseen validation data.

9. Conclusion

This project provided a valuable opportunity to explore and apply different machine learning techniques aimed at improving model performance. Throughout the process, I tested various methods such as data augmentation, regularization, and adjusting model architectures to enhance the model's ability to generalize to unseen data.

By experimenting with different strategies, I was able to achieve a solid performance on Kaggle, with the model demonstrating strong accuracy on both the training and validation sets. While there is always room for further improvement, the results validated the effectiveness of the chosen approach, showing that the model can effectively handle unseen data and avoid overfitting.